

AMC feedback loop

Technical design v2 — end-to-end flow

AMC codebase capture → Human Feedback Loop module (unified record) →
external Evaluation Layer (batch + tiered, repair)

v2 draft · three-system architecture: AMC, Human Feedback Loop module, Evaluation Layer

1. End-to-end flow

Three separate systems are involved. **AMC** owns the query/response flow, the feedback UI panel, and the `feedback_records` table. The **Human Feedback Loop module** is a separate codebase that builds one unified evaluation record per response. The **Evaluation Layer** is a separate system again, triggered per record, that does the actual deep evaluation and repair — its design is covered in Section 4 and is expected to change.

1.1 Capturing feedback in AMC

The response panel offers issue-type checkboxes and a feedback text box next to every AMC response. Active and passive capture both write to the same table; the difference is what triggers the write and how much of the record is populated.

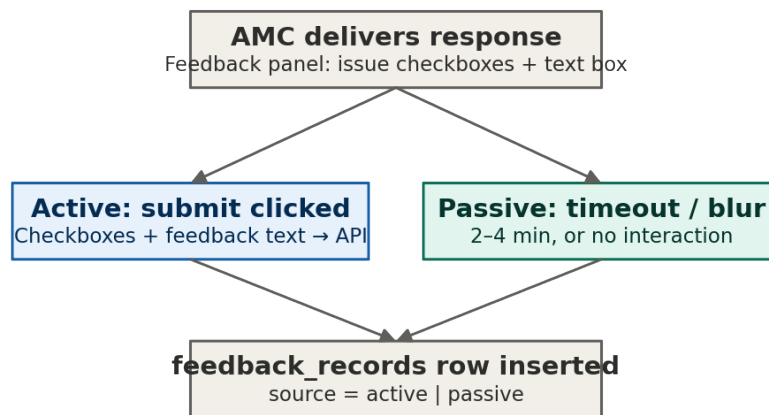


Figure 1 — active and passive capture into `feedback_records`

- **Active:** user checks one or more issue types, optionally adds free text, clicks Submit — sent via the feedback API immediately.
- **Passive:** a 2–4 minute window is given for the user to react. If the window elapses with no submit, or a blur/click-away event fires on the feedback box first, a row is captured with whatever partial state exists (often nothing — the response was accepted silently).

1.2 Handoff to the Human Feedback Loop module

Once a `feedback_records` row exists (from either path), AMC triggers the module — this is an active call, not the module polling AMC's database, since the two are separate codebases and AMC owns the source table.

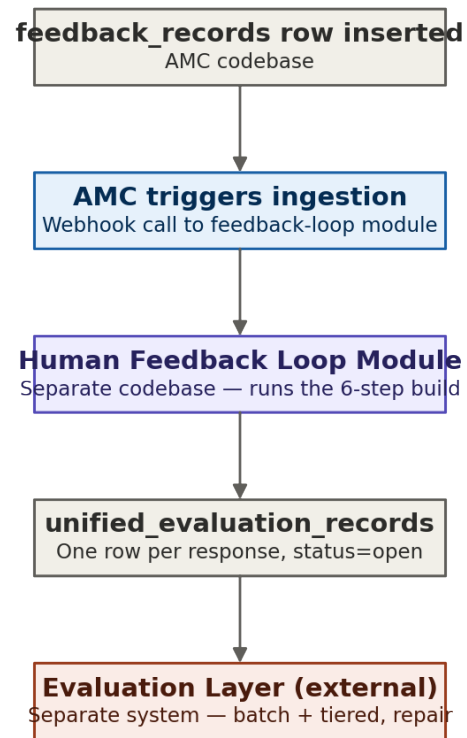


Figure 2 — trigger through to the external evaluation layer

The module runs the same four-step build for both active and passive rows, skipping steps that need data the passive path doesn't have (Section 3). The result is one JSON record in `unified_evaluation_records`, which the external Evaluation Layer then picks up independently.

2. AMC codebase changes

Everything in this section lives in the existing AMC / streamlit_app codebase. Nothing here depends on the Human Feedback Loop module's internals — only on its trigger contract.

2.1 UI panel

- Rendered alongside every delivered response.
- Issue-type checkboxes (multi-select) — e.g. Inaccurate, Incomplete, Irrelevant, Outdated, Other.
- A feedback text box for free-form detail.
- One Submit button. Clicking it is the active-path trigger; the panel also arms the passive-path timer/blur listener the moment the response renders, regardless of whether the user ever touches it.

2.2 feedback_records schema

As specified:

Field	Type	Req.	Description
response_id	str (UUID)	Yes	Links to the original response. Validated against ResponseMetadataCache (24h TTL). 404 if not found.
session_id	str (UUID)	Yes	User session identifier. Used for duplicate detection within a 5-minute window.
user_id	Optional[str]	No	Nullable for anonymous feedback. Role-based — compliance officers tracked explicitly.
rating	RatingType	Yes	Overall satisfaction signal. Positive/negative used as prior weight in quality score.
feedback_type	FeedbackType	Yes	Category of the complaint or correction. Drives evaluation tier selection.
entity_name	Optional[str]	No	The entity the user is correcting (e.g. “Axis Bluechip Fund”). NER also extracts from notes.
entity_type	Optional[str]	No	Taxonomy label: FUND, SCHEME, BENCHMARK, MANAGER, etc.
notes	Optional[str], max 2000	No	Free-text correction or complaint. NER + VADER sentiment run on this field.
corrections	Optional[dict]	No	Structured: {proposed_entity, proposed_value, proposed_relationship}.
timestamp	Optional[datetime]	No	Client timestamp. Falls back to server time if absent.

Proposed additions

Four fields the flow depends on but that weren't in the list above — flagging these as additions rather than assuming them:

Field	Type	Req.	Description
feedback_id	str (UUID), PK	Yes	Row's own identifier. UnifiedEvaluationRecord.feedback_ref points here for HITL entries — nothing currently generates this key.
source	Literal['active','passive']	Yes	Raw capture mode. Distinct from feedback_type (nature of the complaint) and from entry_route (a derived classification computed downstream in Section 3).
received_at	timestampz	Yes	Server-side write time. The 5-minute dedup window needs a clock that can't be skewed or omitted by the client, unlike timestamp.
ingestion_status	Literal['pending','sent','failed']	Yes	Tracks delivery of the trigger call to the Human Feedback Loop module (Section 2.4).

2.3 Where does the initial response live?

Not in either table, and not duplicated. response_id validation against ResponseMetadataCache (24h TTL) is a fast existence check, not the response's storage — the cache almost certainly holds identifiers and light metadata, not the full response payload. The full frozen context (D0–D3, including the response itself) already lives in AMC's interaction_evidence table, which UnifiedEvaluationRecord.evidence_snapshot_ref points to (Section 3). Both tables in this document only ever reference it by response_id / evidence_snapshot_ref.

Worth confirming: if ResponseMetadataCache is the only existence check at submission time and it's a 24h TTL cache, feedback submitted more than 24h after a response would 404 even though the underlying interaction_evidence row is presumably still there. If delayed active feedback (next-day) needs to be supported, the validation check may need to fall back to interaction_evidence directly once the cache entry has expired.

2.4 Trigger to the Human Feedback Loop module

Immediately after the insert (either path), AMC calls the module's ingestion endpoint with just the identifiers — the module fetches the rest itself via its own EvidenceAdapter, keeping AMC's payload minimal and avoiding a second source of truth for the feedback content.

```
POST https://feedback-loop.internal/ingest
{
  "feedback_id": "...",
  "session_id": "...",
  "response_id": "...",
  "source": "active" | "passive"
}
```

Decision to confirm: a webhook call can fail or the module can be briefly down. Recommend AMC sets ingestion_status='pending' on insert, flips to 'sent' on a 2xx response, and a lightweight retry sweep (e.g.

every 5 minutes) re-sends anything still 'pending' or 'failed' after N minutes — so a dropped webhook call doesn't silently lose a feedback row.

3. Building the unified evaluation record

This is the Human Feedback Loop module's only job: turn one `feedback_records` row into one `unified_evaluation_records` row. Four steps run, each adding to the same JSON object; nothing is persisted until the last applicable step finishes. Runs for both `hitl` and `passive` rows, skipping the parts that need text a `passive` row doesn't have.

Parallel or sequential?

Neither, fully — the four steps split into a genuinely independent pair and a strictly dependent pair. **Steps 1 and 2 run concurrently.** Both only need the raw `feedback_records` row as input; step 1 (classification/sentiment) reads notes, step 2 (evidence + enrich/dedupe) reads `response_id/session_id` — neither consumes the other's output, so there's no reason to serialize them. **Steps 3 and 4 must run sequentially**, and after both 1 and 2 finish: step 3's rule check needs `evidence_snapshot_ref`, which only exists once step 2 completes; step 4's adjudication needs step 3's fired-rules output to classify `root_cause` — it cannot run first. Running all four fully in parallel isn't possible (4 would adjudicate nothing), and running all four fully sequentially is safe but leaves free concurrency on the table, since 1 and 2 never touch each other's output.

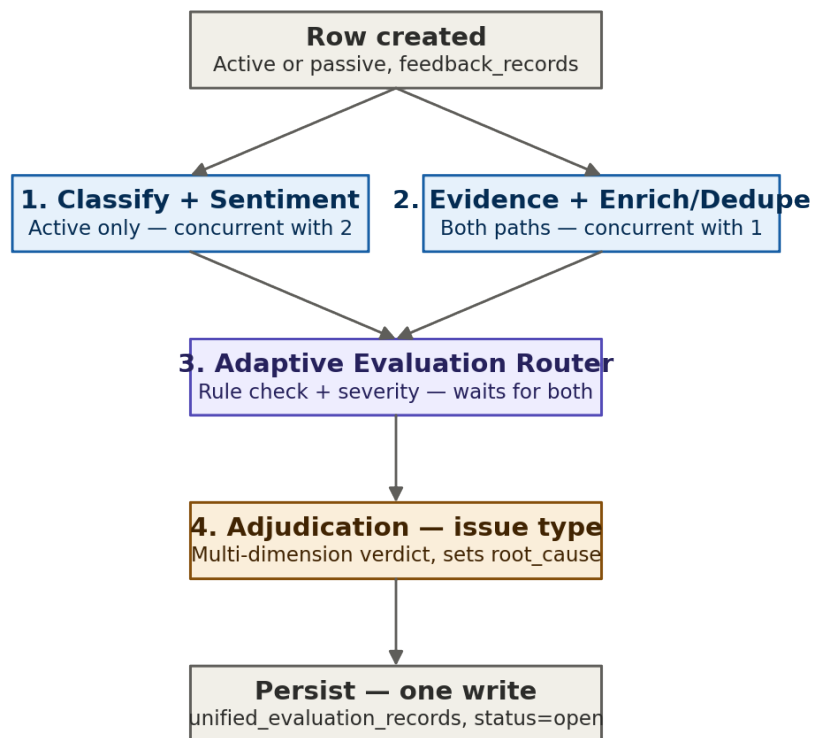


Figure 3 — 1 and 2 run concurrently and join before 3, then 3 → 4 stay sequential

One consequence: step 3 optionally uses step 1's classification (e.g. to focus the rule check on the issue type the user flagged), but can't *require* it, since step 1 may still be in flight or simply skipped on a `passive` row — the join point after 1 and 2 needs to treat step 1's output as optional input to step 3, not a hard

dependency.

What each step does, and what it needs

- **1 — Classification (NER) + Sentiment + Intent:** extracts entities/issue type from notes, entity_name/entity_type, and scores tone/intent on the same text. Skipped when corrections is already structured — nothing to extract if the proposed entity/value/relationship is already explicit. *Active only* — no text on a silent passive row.
- **2 — Fetch the Evidence Pack + Enrich + Deduplicate:** pulls the frozen D0–D3 snapshot for this response_id from AMC's interaction_evidence table via the read-only EvidenceAdapter and sets evidence_snapshot_ref; clusters this feedback against other open feedback on the same response/entity (a duplicate match short-circuits the rest); and derives entry_route: hitl if source='active', otherwise dissatisfaction_converted if an implicit negative signal fires (below), else plain automatic. Runs for *both* paths.
- **3 — Adaptive Evaluation Router (rule check + severity):** runs a deterministic rule check against the evidence and any structured corrections, and maps whichever rules fire to a preliminary severity (critical/high/medium/low) — a compliance or jurisdiction rule violation triages as critical, a presentation-only mismatch as low. Runs for *both* paths.
- **4 — Adjudication (multi-dimension verdict, issue type):** reasons across several dimensions of the evidence — and classification, when available — to classify root_cause (knowledge, retrieval, graph, context_engineering, model, guardrail, compliance, presentation, or feedback_invalid), and sets lifecycle_status='open'. This step does not decide *how* to fix anything or run any tier — it only hands off a triaged record for the Evaluation Layer to pick up.

Worth confirming: step 3's rule check and the Evaluation Layer's Tier 1A (Section 4.2) both run deterministic rule checks against the same evidence. As specified, these are separate executions — step 3's result is for triage only and isn't written to deterministic_results, which stays an Evaluation-Layer-owned field. That's some duplicated work; worth a follow-up on whether the two should share one rule engine invocation, or genuinely need to stay independent (e.g. because the module's check is meant to be a coarser/faster subset than the Evaluation Layer's authoritative one).

Active vs. passive: what actually runs

Step	hitl (active)	automatic / dissatisfaction_converted (passive)
1. Classification + Sentiment/Intent	Runs	Skipped — no text
2. Fetch Evidence + Enrich + Dedupe	Runs	Runs — also decides automatic vs. dissatisfaction_converted
3. Adaptive Router (rule check + severity)	Runs	Runs
4. Adjudication (issue type)	Runs	Runs (evidence-only)

A dissatisfaction_converted row is still passively captured — no text was submitted — so it skips step 1 exactly like a plain automatic row. The conversion only changes how the row is classified and prioritised, not what data is available to step 1.

Dissatisfaction signal detection

This is the logic step 2 uses to decide `automatic` vs. `dissatisfaction_converted` on a passive row. Silence alone (no follow-up at all within the window) stays `automatic` — there's nothing to infer dissatisfaction from. The signal only fires when the user's *next* query in the same session, within the same passive window (2–4 min), looks like a reaction to a bad answer rather than a new question.

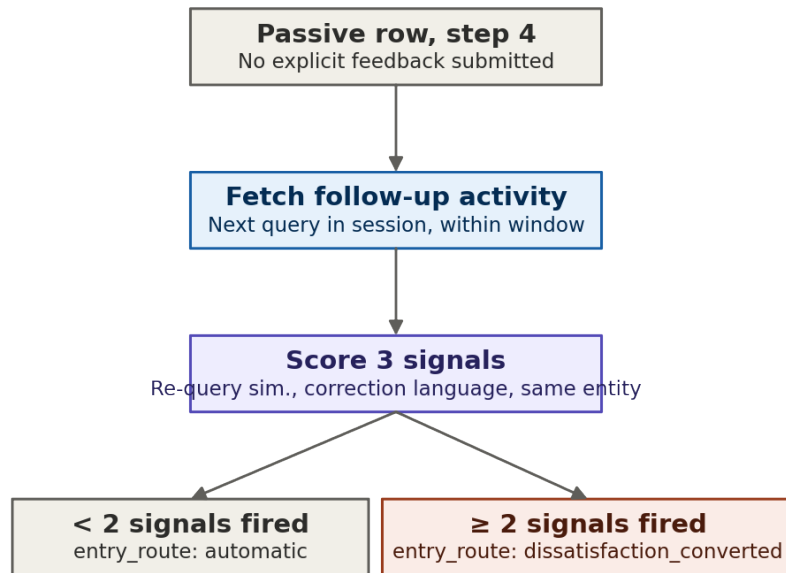


Figure 3b — scoring implicit dissatisfaction on a silent passive row

Three deterministic, zero-token signals, each computed against the follow-up query text:

- **Re-query similarity:** cosine similarity between the original query and the follow-up ≥ 0.75 — the user is essentially asking the same thing again, which usually means the first answer didn't land.
- **Correction language:** the follow-up matches a small negation/correction pattern set ("no, that's wrong", "actually it's...", "that's not right", "should be...") — a lightweight regex/keyword check, not a model call.
- **Same entity:** NER on the follow-up (the same tooling step 1 uses, just re-run on a different field) extracts the same `entity_name` the original response was about.

Requiring **at least two of three** is deliberate: an ordinary follow-up question ("and what about last year's returns?") will often trip *same entity* alone, and a genuinely unrelated new question can coincidentally score high on similarity. Two independent signals agreeing is a materially stronger claim than either alone — the same reasoning the confidence gate elsewhere in this design uses.

```
def detect_dissatisfaction_signal(row, evidence_snapshot_ref, adapters, config):  
    # returns DissatisfactionSignal | None  
    followups = adapters.evidence.get_followup_activity(  
        row.session_id, row.response_id, window_minutes=config.passive_window_minutes)  
    if not followups:  
        return None # silence – nothing to infer, stays automatic
```

```

next_query = followups[0]
fired = []

similarity = semantic_similarity(evidence_snapshot_ref.original_query, next_query.text)
if similarity >= config.requery_threshold:
    fired.append("requery_similarity")
if matches_correction_pattern(next_query.text):
    fired.append("correction_language")
if evidence_snapshot_ref.entity_name in extract_entities(next_query.text):
    fired.append("same_entity")

if len(fired) >= config.min_signals_required: # default: 2
    return DissatisfactionSignal(triggers=fired, source_text=next_query.text,
                                confidence=len(fired) / 3)

return None

```

Thresholds above (0.75 similarity, 2-of-3) are starting points, not calibrated numbers — they need validation against real session data before launch, and should be config-driven (config.requery_threshold, config.min_signals_required) so they can be tuned without a redeploy. Also new: this requires an EvidenceAdapter.get_followup_activity() method AMC needs to expose — the module doesn't otherwise have visibility into in-session query activity beyond the one response it's evaluating.

For auditability, worth adding one more field to UnifiedEvaluationRecord beyond what's specified: dissatisfaction_evidence: Optional[dict], storing which signals fired and the follow-up text that triggered them — otherwise a dissatisfaction_converted row carries no explanation of why it was converted, which will matter the first time someone questions a triage decision.

The function that builds the record

The module only ever writes the first eight fields below. The remaining six (deterministic_results through adjudication_confidence) stay null until the Evaluation Layer updates this same row — Section 4 covers that half.

```
class UnifiedEvaluationRecord(BaseModel):
    # --- set by the Human Feedback Loop module (this section) ---
    session_id: str                                # UUID, original query interaction
    response_id: str                                # UUID -> evidence snapshot
    entry_route: Literal["hitl", "automatic", "dissatisfaction_converted"]
    feedback_ref: str | None = None                 # feedback_id, if route==hitl
    evidence_snapshot_ref: str                      # FK -> interaction_evidence
    root_cause: Literal["knowledge", "retrieval", "graph", "context_engineering",
                        "model", "guardrail", "compliance", "presentation",
                        "feedback_invalid"] | None = None    # step 4
    severity: Literal["critical", "high", "medium", "low"] | None = None    # step 3
    lifecycle_status: Literal["open", "diagnosing", "resolved", "monitoring"] = "open"

    # --- set later by the Evaluation Layer (Section 4) ---
    deterministic_results: dict | None = None        # JSONB, Tier 1A: all 9 rule categories
    gnn_plausibility_score: float | None = None      # Tier 1B
    semantic_results: dict | None = None             # JSONB, Tier 2: NLI scores
    llm_evaluation: dict | None = None               # JSONB, only if Tier 3 reached
    adjudication_verdict: Literal["valid", "partially_valid", "invalid",
                                   "subjective", "insufficient_evidence"] | None = None
    adjudication_confidence: float | None = None     # confidence across tiers run

def build_unified_record(row, adapters, db) -> UnifiedEvaluationRecord | None:
    # Steps 1 and 2 run concurrently – neither depends on the other's output.
    # Step 1 is skipped up front (not scheduled at all) rather than launched and discarded.
    with concurrent.futures.ThreadPoolExecutor(max_workers=2) as pool:
        step1 = None
        if row.source == "active" and row.notes and not row.corrections:
            step1 = pool.submit(run_classification, row.notes, row.entity_name, row.entity_type)
        step2 = pool.submit(run_evidence_and_enrichment, row, adapters, db)

        classification, sentiment_intent = step1.result() if step1 else (None, None)
        evidence_snapshot_ref, enrichment = step2.result()

    if enrichment.is_duplicate:
        return None    # already covered by an existing cluster

    if row.source == "active":
        entry_route = "hitl"
    elif enrichment.dissatisfaction_signal:
        entry_route = "dissatisfaction_converted"
    else:
        entry_route = "automatic"

    # Step 3 – needs step 2's evidence; step 1's classification is optional, not required
    rules_fired = run_rule_check(evidence_snapshot_ref, row.corrections)
    severity = severity_from_rules(rules_fired)

    # Step 4 – needs step 3's output; cannot run before it
```

```
root_cause = adjudicate_issue_type(evidence_snapshot_ref, rules_fired, classification)
```

```
return UnifiedEvaluationRecord(  
    session_id=row.session_id, response_id=row.response_id, entry_route=entry_route,  
    feedback_ref=row.feedback_id if entry_route == "hitl" else None,  
    evidence_snapshot_ref=evidence_snapshot_ref, root_cause=root_cause,  
    severity=severity, lifecycle_status="open",  
)
```

```
def on_ingest(feedback_id, session_id, response_id, source, adapters, db):  
    """Called by AMC's webhook, or by the retry sweep for a failed delivery."""  
    row = db.fetch_feedback_record(feedback_id)  
    record = build_unified_record(row, adapters, db)  
    if record:  
        db.write_unified_record(record)    # single write, one row per response
```

4. Evaluation layer (evaluation and repair)

This section reflects IMPLEMENTATION_STEPS_BATCH_TIERED.html as supplied and is expected to change. It's included here to show the intended contract with Section 3's output, not as a finalized spec.

Schema reference

Field	Description
session_id	UUID linking back to the original query interaction
response_id	UUID linking to the frozen evidence snapshot (D0–D3)
entry_route	hitl automatic dissatisfaction_converted
feedback_ref	Optional feedback_id if HITL entry
evidence_snapshot_ref	FK to interaction_evidence — the frozen context used at response time
deterministic_results	JSONB — outputs from all 9 rule category evaluators
gnn_plausibility_score	Float from frozen GNN model (0.5 stub in Phase 2)
semantic_results	JSONB — NLI entailment/contradiction/neutral scores
llm_evaluation	Optional JSONB — only populated when Tier 3 was reached
adjudication_verdict	valid partially_valid invalid subjective insufficient_evidence
adjudication_confidence	Float — combined confidence from tiers that ran
root_cause	knowledge retrieval graph context_engineering model guardrail compliance presentation feedback_invalid
severity	critical high medium low
lifecycle_status	open diagnosing resolved monitoring

This system is managed outside the Human Feedback Loop module's codebase, but writes to the *same table* Section 3 writes to — not a separate one. Section 3 inserts a row with `lifecycle_status='open'` and the first eight fields populated; this layer picks up rows in that state, sets `lifecycle_status='diagnosing'`, and *updates* the same row with the six tier/adjudication fields as it works through the ladder below. It owns two engines: a passive-path batch LLM evaluator, and an active/HITL tiered ladder that escalates only when a cheaper tier can't resolve the claim.

4.1 Batch Evaluation Engine (passive path)

Scores feedback in batches of 25 under seven token-optimisation layers, and proposes graph corrections as a validated Cypher mutation rather than applying anything directly.

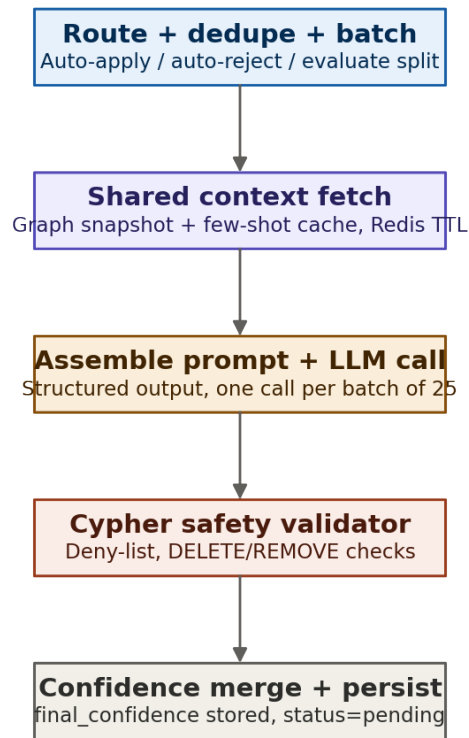


Figure 4 — batch evaluation engine pipeline

- **Route:** quality score ≥ 0.85 and inter-annotator agreement ≥ 0.90 auto-applies without an LLM call; below 0.20 auto-rejects; everything else queues for batching.
- **Dedupe + batch:** embeds and clusters claims (cosine ≥ 0.88), batches the deduplicated queue in groups of 25.
- **Shared context:** one Neo4j round-trip per batch (1-hop subgraph, Redis-cached 1h TTL) and a few-shot example cache (24h TTL) — both amortised across the whole batch instead of per item.
- **Cypher safety validator:** deny-lists DROP/CREATE INDEX/CREATE CONSTRAINT, rejects unguarded DELETE, rejects REMOVE on compliance-sensitive fields. A rejection is marked `insufficient_evidence` and logged, never silently dropped.
- **Confidence merge:** $\text{adjudication_confidence} = \text{llm_confidence} \times \text{inter_annotator_agreement}$, written back onto the same `unified_evaluation_records` row alongside `adjudication_verdict` and `llm_evaluation` — stored regardless of score; the apply-time gate is a separate decision made later.

4.2 Tiered Evaluation Engine (active / HITL path)

A four-tier ladder, cheapest first. Execution stops at the first tier that resolves the claim — a more expensive tier never runs once a cheaper one is confident.

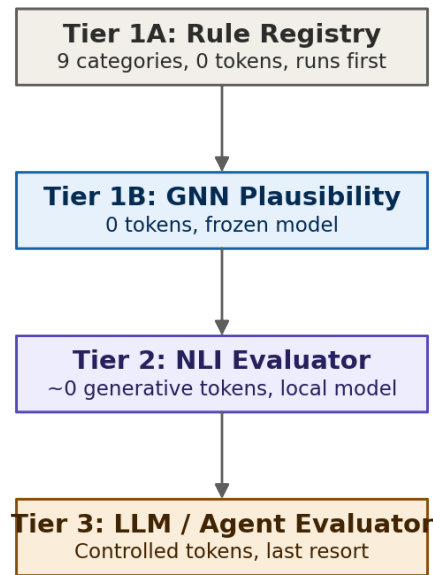


Figure 5 — the four-tier evaluation ladder

Tier	What it does	Cost
1A — Rule Registry	9 deterministic categories (schema, cardinality, referential, temporal, numeric, cross-source, duplicate, hierarchy, jurisdiction)	0 tokens
1B — GNN Plausibility	Structural plausibility score from a frozen model; ships as a 0.5 stub until trained	0 tokens
2 — NLI Evaluator	Local cross-encoder; entailment > 0.7 resolves, contradiction/neutral > 0.7 escalates	~0 generative tokens
3 — LLM / Agent Evaluator	Minimal evidence packet (claim + top-2 chunks + prior tier findings), last resort	Controlled tokens

Escalates to human review when all three scored tiers return confidence below 0.60, the attribute is compliance-sensitive, or two authoritative sources conflict.

Tier 1B build options

Both share the same `GNNPlausibilityScorer.score()` interface, so the choice is an internal swap. Recommended starting point: Option 2 (GDS), since it needs zero new runtime dependencies and runs inside the Neo4j instance already in place; swap to Option 1 later if finer control over the architecture is needed.

	Option 1 — Python R-GCN	Option 2 — Neo4j GDS
Cost	Free (open-source)	Free (GDS Community Edition)

New infra	New PyTorch dependency + offline training job	None — runs inside existing Neo4j
Control	Full — custom architecture, features, loss	Bounded to the GDS pipeline API
Team lift	Requires PyTorch/PyG familiarity	Requires Cypher + GDS familiarity

Appendix: edge cases and open decisions

- **Repair locking (TBD):** if the same underlying issue is flagged by multiple concurrent records while a fix is already in progress, later repair attempts need to be locked or merged against the in-flight one rather than racing it. Locking key, scope (entity-level? recipe-level?), and timeout are not yet designed.
- **Dissatisfaction signal thresholds (need calibration):** the detection logic is defined in Section 3 (re-query similarity, correction language, same entity — 2-of-3 required), but the 0.75 similarity cutoff and the pattern set for “correction language” are starting points that need validation against real session data.
- **Trigger delivery guarantee:** the webhook-plus-retry-sweep approach in Section 2.4 is a recommendation, not yet confirmed — an alternative is a message queue between AMC and the module for at-least-once delivery without a custom retry sweep.
- **Feedback window vs. cache TTL:** flagged in 2.3 — ResponseMetadataCache's 24h TTL may cap how late active feedback can be validated and accepted.
- **Evaluation Layer contract stability:** Section 4 is explicitly subject to change; the contract Section 3 depends on is the shared row shape itself — Section 3 inserts with `lifecycle_status='open'`, Section 4 updates the same row — worth keeping that handoff stable even as the engine internals evolve.